ІНЖЕНЕРНА КНИГА

# Зовнішні результати

**Код. Дослідження. Зовнішній наслідок.**

DiadkoShmek

Власні злиті PR: 16; внески у злиті рішення мейнтейнерів: 4; технічні аудити: 1. Змагання: часових записів 3, подій 2; Zoria враховано один раз.

Датований запис із публічними першоджерелами.

# Зміст

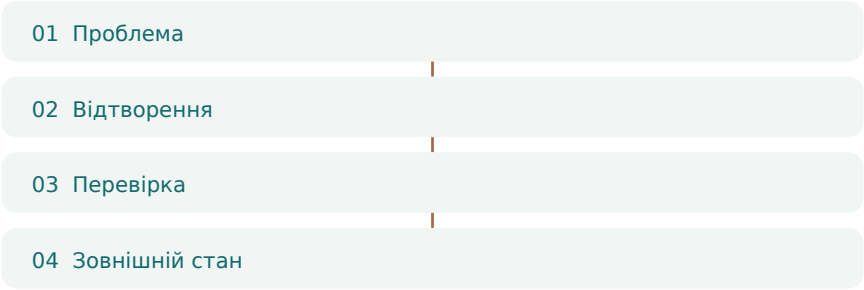
Роль, метод і одиниця доказу . . . . . 4
Поточний портфель PR . . . . . 5
01 OTT-JAX . . . . . 7
02 ONNX IR · graph iterator . . . . . 9
03 ONNX IR · Dropout CSE . . . . . 11
04 Instructor . . . . . 13
05 PyLops . . . . . 15
06 Redis Python client . . . . . 17
07 Microsoft Semantic Kernel . . . . . 19
08 ONNX Range . . . . . 21
09 ONNX IR . . . . . 23
10 FilesDrop . . . . . 25
11 Prime Intellect . . . . . 27
12 M7 MCP . . . . . 29
13 Braintrust SDK JavaScript . . . . . 31
14 Kraft . . . . . 33
15 PyTorch TensorDict . . . . . 35
16 BlackJAX . . . . . 37
17 TensorLy . . . . . 39
18 Apache TVM-FFI . . . . . 41
19 torchode · backsolve adjoint . . . . . 43

20 Lineax · reshaping identities . . . . . 44
21 NumPyro · GaussianStateSpace variance . . . . . 45
22 CodinGame - Mad Pod Racing . . . . . 46
23 Zoria 2026 / UA SKILLS . . . . . 48
Як зростає журнал . . . . . 50

# Роль, метод і одиниця доказу

Я досліджую дефект, визначаю контракт і перевіряю наслідок. AI-інструменти допомагають із пошуком, реалізацією та перевіркою; я задаю напрям, відповідаю за інженерне рішення й погоджую зовнішні дії. Ця книга не вимірює відсоток автономності чи перевагу однієї моделі.

Власні злиті PR: 16; внески у злиті рішення мейнтейнерів: 4; технічні аудити: 1. Змагання: часових записів 3, подій 2; Zoria враховано один раз.




Локальний тест перевіряє механізм. Upstream merge або credit підтверджує зовнішній наслідок. Жоден із них сам собою не доводить повний CI, adoption чи оплату.

Дати мають UTC-прив’язку. Для історичних зрізів збережені початкові часові поля; дата перевірки оголошення може відрізнятися від дати публікації.

# Поточний портфель PR

Публічні PR у чужих репозиторіях: 134 подано; 16 злитих, 108 відкритих, 10 закритих без merge. Зріз GitHub: 2026-11-11 10:34 UTC. Власна та приватна активність виключені.

Цей зріз відділяє завершені результати від відкритої роботи. Відзначені внески в реалізації мейнтейнерів обліковуються окремо від власних merge; закриті без merge PR не прирівнюються до відмов.

### Instructor #2899 / our #2866

Наш #2866 включений у merged #2899 із прямим посиланням; співавторський рядок є у PR head, але відсутній у фінальному squash-коміті

### OTT-JAX #716

Прямий merge 05.11.2026 о 21:21:59 UTC; michalk8 скоротив регресію, production dispatch fix збережений; 9 hosted check runs SUCCESS на 60222b14

### ONNX IR #533

Прямий merge 05.11.2026 о 21:15:56 UTC через justinchuby; структурна регресія збережена, thread-pool helper edits прибрані

### ONNX IR #531

Мейнтейнер схвалив попередню версію й попросив параметризувати регресію через pytest. Це враховано в коміті 661c72f

### Docling Core #953

Відкритий; GitHub показує REVIEW_REQUIRED. Поточні CI та всі правила merge цим зрізом не засвідчуються

### Triton #12611

Відкритий інженерний кейс; upstream merge не підтверджено

## ONNX #8734

Прямий merge підтверджено свіжим повним переліком PR

## Semantic Kernel #14701

Прямий merge 04.11.2026 о 14:46:53 UTC підтверджено

## Redis #4550

Прямий merge 04.11.2026 о 15:44:00 UTC підтверджено

## PyLops #837

Прямий merge підтверджено; адаптація тесту мейнтейнером збережена в історії кейсу

## ONNX IR #529

Прямий merge 05.11.2026 о 21:21:07 UTC через justinchuby; iterator graph I/O slice fix

## NumPyro #2405

Злито 9 листопада. Виправлено розташування часової осі дисперсії для кількох батч-осей

## NumPyro #2410

Гілка 6e84563 містить виправлення #2405 з master, тест двовимірного батчу й опис змін форм

Повний датований перелік PR

Зріз не прогнозує merge і не засвідчує всі перевірки CI. Triton #12611 поданий; до злитих результатів він не входить.

# 01 OTT-JAX

**2026-11-05 · Власний PR merged · UTC**
**2026-11-05T21:46:10.662Z · 2026-11-05T21:21:59Z**

## Проблема

Документований ImplicitDiff(solver=...) приймав користувацький callback, але differentiation падала з UnboundLocalError до його виклику. Локальний solver отримував значення лише у default-гілці.

## Рішення та роль

Внесок ініціалізує локальний solver перед існуючим explicit-None fallback. Типова схема, Schur/transpose logic і kwargs forwarding збережені. Мейнтейнер michalk8 скоротив регресію до одного public Sinkhorn cost-gradient порівняння.

## Перевірка

Наш #716 злитий через michalk8 05.11.2026 о 21:21:59 UTC, commit 8683e37d162db8b1fbb746afc28825c938f6e833. На reviewed head 60222b14 підтверджені дев’ять hosted check runs SUCCESS, включно з Lint і Tests. Поточний тест використовує use_danskin=False, порівнює custom/default x-gradients і перевіряє рівно один виклик із потрібними kwargs. Ширші попередні локальні перевірки збережені як історичні; під час оновлення книги їх не повторювали.

## Що переношу далі

Ремонт extension-point має відновлювати виклик callback у реальному споживачі. Після maintainer pruning треба описувати фактичний фінальний тест і CI, не переносити на нього ширше історичне покриття.

## Межа зовнішнього результату

Прямий merge із maintainer-pruned тестом. Поточна регресія перевіряє n<=m/default-LSE dispatch path, не довільний solver. Дев’ять check runs прив’язані до reviewed head; локального повного runtime re-audit, GPU/higherAD/performance або production adoption не заявлено.

## Джерела

1. Першоджерело

2. Докладний кейс

# 02 ONNX IR · graph iterator

**2026-11-05 · Власний PR merged · 2026-11-05T21:21:07Z**

## Проблема

Присвоєння finite iterator/generator у graph.inputs[:] або graph.outputs[:] спочатку витрачало його на ownership tracking, а потім передавало порожній iterable у list assignment. Clone міг втратити graph I/O, а extended slices - помилково відхиляти коректну довжину.

## Рішення та роль

Заміна матеріалізується в tuple один раз до обох споживачів. Це узгоджено з constructor/extend pattern; ownership і Python slice semantics лишаються в наявному власнику.

## Перевірка

Наш #529 злитий через justinchuby 05.11.2026 о 21:21:07 UTC, commit 045f02a0dad7c8135305e4f5022784d4550d40e7. Фінальні два файли зберігають tuple snapshot і graph/model regressions. Історичний PR report має 760 tests/104 subtests та окремий 239/96 review; це не прогін під час цього оновлення книги.

## Що переношу далі

Одноразовий iterable треба зафіксувати перед кількома споживачами. Перевірка серіалізації та повного ONNX model checking показує наслідок втрати graph I/O за межами контейнера.

## Межа зовнішнього результату

Прямий merge нашого PR. Finite iterable slices; rollback некоректних операцій не розширений. Runtime counts - датований PR report, не повторний прогін.

## Джерела

1. Першоджерело

2. Докладний кейс

# 03 ONNX IR · Dropout CSE

**2026-11-05 · Власний PR merged · 2026-11-05T21:15:56Z**

## Проблема

CSE об’єднував незалежні standard-domain Dropout evaluations з однаковими входами. Їхні sampled masks ставали одним значенням, і downstream Xor перетворювався на Xor(v,v), тотожно false.

## Рішення та роль

Dropout додано до наявного переліку nondeterministic exclusions. Регресія структурно зберігає два вузли й різні Xor operands, без вимоги, щоб дві випадкові маски обов’язково відрізнялися. Зайві thread-pool зміни тестового helper прибрані.

## Перевірка

Наш #533 злитий через justinchuby 05.11.2026 о 21:15:56 UTC, commit 4b0302491a601f445e9035097d0b9f6f0f7f90fe. Exact final head ab1253c9 містить лише exclusion і структурний test. Останній amendment документує 12 passing CSE methods; source/runtime proof вже виконаний раніше, під час підтримки книги його не повторювали.

## Що переношу далі

CSE має враховувати семантику випадкового evaluation, а не лише тотожність входів. Структурна регресія перевіряє незалежність вузлів без flaky припущення про конкретні random samples.

## Межа зовнішнього результату

Прямий merge нашого PR. Структурний Dropout test не вимагає нерівності випадкових масок. Seeded/inference paths теж conservatively excluded; 12 тестів - попередній amendment proof, не новий прогін.

## Джерела

1. Першоджерело

2. Докладний кейс

# 04 Instructor

**2026-11-05 · Внесок у merged fix мейнтейнера**

## Проблема

Sanitizer часткового JSON у streaming output видаляв DEL/C1 навіть усередині валідних рядків. Значення й ключі могли змінитися до завершення валідації.

## Рішення та роль

Наша пропозиція #2866 звузила regex до C0 й додала Unicode streaming regressions. Мейнтейнер консолідував її з іншими пропозиціями в #2899: per-stream tracking лапок і escape-послідовностей зберігає DEL/C1 у JSON-рядках та історичне repair поза ними.

## Перевірка

Native GitHub підтверджує merge #2899 через jxnl-oai 05.11.2026 о 18:47:01 UTC, commit 5f352b0085fb2f8adaa26d69a6447ed5cabd2034. Head 723629d57c05edb996db411e93e7b37ae5c95a8b збігається з перевіреним source audit: C0-only return AST збережений, sync/async тести розширені. 101 passing tests — звіт мейнтейнера; під час оновлення книги їх не повторювали.

## Що переношу далі

Власний PR і merged maintainer consolidation — різні наслідки. Атрибуцію треба звіряти і на reviewed head, і на фінальному squash commit: trailer може зникнути, навіть коли явний reference зберігся.

## Межа зовнішнього результату

Наш #2866 закритий без merge. #2899 злитий із явним посиланням на #2866; coauthor trailer є на head, але відсутній у фінальному squash. Це четвертий incorporated contribution, не одинадцятий direct merge. Runtime/provider checks не повторювали.

## Джерела

1. Першоджерело

2. Наша пропозиція

3. Відзначення внеску

4. Докладний кейс

# 05 PyLops

**2026-11-04 · Власний PR merged · UTC**
**2026-11-04T21:44:06.308045+00:00 · 2026-11-04T21:34:47Z**

## Проблема

Щільна матриця нестаціонарної одновимірної згортки містила імпульсні відгуки рядками замість стовпців. Множення й пряме обернення розходилися з публічним оператором навіть для симетричних фільтрів, що змінюються вздовж сигналу.

## Рішення та роль

Внесок транспонує зібрану матрицю та перевіряє її forward і adjoint дії в наявному seeded тесті. Matteo Ravasi перед merge переставив перевірки навколо Cdense; виробничий fix залишився тим самим.

## Перевірка

Native GitHub підтверджує прямий merge #837 о 21:34:47 UTC 04.11.2026. PR документує 18 passing nonstationary tests, 543 neighboring passes/3 skips, 128 coordinate-oracle матриць і 48 operator-action cases. У конкретній 51-sample задачі dense-inverse error зменшився з 8.24% до менш ніж 2e-16. Runtime під час оновлення книги не повторювали.

## Що переношу далі

Матриця оператора має відтворювати його дію на координатних імпульсах. Перевірка і forward, і adjoint відділяє помилку орієнтації від випадково симетричного контрольного прикладу.

## Межа зовнішнього результату

Прямий merge і maintainer adaptation підтверджено. Лише одновимірний дійсний оператор; старі multidimensional/complex обмеження збережені. Runtime не повторювали, production-використання не встановлено.

## Джерела

1. Першоджерело

2. Докладний кейс

# 06 Redis Python client

**2026-11-04 · Власний PR merged · UTC**
**2026-11-04T16:20:18.445505Z · 2026-11-04T15:44:00Z**

## Проблема

Якщо вхід до MONITOR context завершувався помилкою, Python не викликав exit і позичене з’єднання лишалося зайнятим. Пул max_connections=1 міг перестати обслуговувати навіть PING; async setup також має шлях скасування.

## Рішення та роль

Sync/async MONITOR звільняють з’єднання після невдалого входу, від’єднують його від Monitor до exit cleanup та повертають до пулу через finally. Async release захищено shield. Після рев’ю прибрано зайві exception-precedence гілки й варіації, зберігши потрібний maintainer shape.

## Перевірка

Опис PR фіксує перевірку на тимчасовому loopback Redis 8.0.2: 23 passing і 3 native skips для RESP2 та окремо RESP3, включно з ACL denial, повторним використанням одноз’єднаннєвого пулу та async cancellation. GitHub підтверджує прямий merge #4550. Сервер під час оновлення книги не запускався.

## Що переношу далі

Невдалий context entry не запускає автоматичний exit. Власник позиченого ресурсу має явно відновити пул і перевірити його реальною наступною командою.

## Межа зовнішнього результату

Наш PR злитий upstream. Раніше зафіксовані локальні тести під час оновлення книги не повторювалися; повне production-використання не встановлено.

## Джерела

1. Першоджерело

2. Докладний кейс

# 07 Microsoft Semantic Kernel

**2026-11-04 · Власний PR merged · UTC**
**2026-11-04T16:20:18.445505Z · 2026-11-04T14:46:53Z**

## Проблема

Після раннього закриття або скасування streaming invocation вихідний sync/async Python generator лишався призупиненим. Його finally не звільняв ресурси в момент завершення виклику; утриманий reference робив це незалежним від garbage collection.

## Рішення та роль

Внесок обгортає реальні native generators у closing/aclosing та закриває створений function-invocation stream у Kernel.invoke_stream. Звичайні значення й custom close-bearing результати не отримують чужого cleanup; публічні сигнатури збережено.

## Перевірка

Публічний PR документує 8 failing і 148 passing тестів на базі та 156 passing на кандидатові, 18 доданих сценаріїв із утриманими генераторами й окремі перевірки скасування. GitHub підтверджує прямий merge #14701. Під час оновлення книги runtime-тести не повторювалися.

## Що переношу далі

Той шар, який володіє створеним потоком, має гарантовано завершувати його. Утриманий reference у регресії відділяє явний cleanup від випадкового прибирання GC.

## Межа зовнішнього результату

Наш PR злитий upstream. Раніше зафіксовані локальні тести під час оновлення книги не повторювалися; повне production-використання не встановлено.

## Джерела

1. Першоджерело

2. Докладний кейс

# 08 ONNX Range

**2026-11-04 · Власний PR merged · 2026-11-04T01:09:20Z**

## Проблема

Integer Range shape inference віднімав start від limit у знаковому типі до перетворення на floating point. Для INT32 від −2147483648 до 2147483647 із кроком 2147483647 переповнення давало нульову довжину замість трьох; floating-point ділення також може округлити частку INT64 за межу стелі.

## Рішення та роль

Спільний helper для обох історичних схем обчислює беззнакову відстань і модуль кроку, а потім точне цілочисельне ділення з округленням угору. Підтримуються обидва напрями та мінімальний від’ємний крок; floating-point шлях не змінено.

## Перевірка

GitHub підтверджує прямий merge PR #8734 4 листопада 2026 року та merge commit. Опис PR документує приклад і підхід; runtime-тести під час оновлення книги не повторювалися.

## Що переношу далі

Для дискретної кількості елементів точна цілочисельна арифметика з безпечним представленням відстані усуває і переповнення знакової різниці, і помилку округлення частки.

## Межа зовнішнього результату

Прямий merge і upstream approvals підтверджено; runtime-тести під час оновлення книги не повторювали, production-використання не встановлено.

## Джерела

1. Першоджерело

2. Докладний кейс

# 09 ONNX IR

**2026-11-03 · Власний PR merged · UTC**
**2026-11-03T17:33:18.542858Z · 2026-11-03T16:40:54Z**

## Проблема

У графі y = Relu(x), z = Identity(y) обидва імені y та z є публічними виходами. Усунення Identity перейменовувало перший вихід на z, залишаючи два z. Запит за початковим іменем у ставав недійсним, хоча локальна форма графа могла пройти перевірку.

## Рішення та роль

Identity зберігається, коли його вхід уже є публічним виходом або належить зовнішньому графу. Другий випадок захищає захоплені батьківські значення від перейменування та перенесення у виходи підграфа. Безпечні внутрішні aliases і далі усуваються.

## Перевірка

В описі поданого PR зафіксовано 28 focused tests і 9 subtests, перевірки іменованих виходів через ONNX ReferenceEvaluator та ONNX Runtime CPU і NumPy oracle. До подання AI-рев’ю перевірило ще 23 сценарії та виявило потребу захистити власність батьківського значення. Для оновлення книги runtime-тести не повторювалися; merge commit і час звірено безпосередньо з GitHub.

## Що переношу далі

Оптимізація графа має зберігати публічні імена та власність значень, а не лише числовий результат усередині графа.

## Межа зовнішнього результату

Наш PR #521 злитий у main. Збережені focused CPU-перевірки не підміняють повну матрицю nox, Python, Torch або GPU. Оновлення журналу підтверджує merge, а не повторний запуск усіх тестів чи production-використання.

## Джерела

1. Першоджерело

2. Докладний кейс

# 10 FilesDrop

**2026-10-21 · Власний PR merged**

## Проблема

Я почав із двох різних порушень життєвого циклу: невдале видалення зникало з наступної партії cleanup, а deadline не припиняв очікування DNS. У першому випадку важлива не сама помилка диска, а втрата можливості повторити роботу. У другому - межа між очікуванням запиту та системним resolver.

## Рішення та роль

Я зберіг невдалі елементи для наступного проходу й відділив успішні видалення. Контрприклад на 501 файл показав, що одна постійна відмова може блокувати наступну партію. Для URL-upload сигнал скасування обмежує очікування, зберігаючи перевірки адрес та redirect-поведінки.

## Перевірка

Регресії спершу падали на базі. У збереженому локальному прогоні пройшли 236 тестів у 12 API suites. Тимчасові файли та DNS/HTTP doubles перевіряли потрібні переходи стану, не поведінку production-сервісу.

## Що переношу далі

Cleanup має зберігати можливість повторення. Скасування очікування не означає зупинку системної операції.

## Межа зовнішнього результату

Злиті API-виправлення з локальними регресійними перевірками. Розгортання у робочому середовищі, перевірка живого сервісу чи оплата цим записом не підтверджені.

## Джерела

1. Першоджерело

2. Докладний кейс

# 11 Prime Intellect

**2026-10-24 · Внесок у merged fix мейнтейнера**

## Проблема

Я досліджував, як конфігурація моделі й клієнта успадковує загальні sampling-параметри. Інший учасник уже подав початковий issue. Моя задача полягала в тому, щоб перетворити опис на перевірюваний контракт і зрозумілий відтворювач.

## Рішення та роль

Я запропонував межу на рівні явного model/client override. Мейнтейнер обрав точнішу межу клієнта: інша модель на тому самому сервері зберігає спільні параметри, а окремий клієнт отримує власні. Це інша реалізація й дещо інший контракт, а не merge мого патчу.

## Перевірка

Матриця моєї пропозиції перейшла від 11 failing і 7 passing сценаріїв до 18 passing. Це число сценаріїв, не кількість дефектів. Публічна відповідь мейнтейнера окремо визнає роль відтворення й дослідження. Живе inference або навчання я цими тестами не підтверджував.

## Що переношу далі

Корисний внесок може змінити рішення мейнтейнера, навіть коли власний patch не стає прийнятим кодом.

## Межа зовнішнього результату

Мейнтейнер відзначив наше відтворення й дослідження. Наш патч замінено, а не злито; початкову проблему зареєстрував інший учасник. Працевлаштування або оплата не заявляються.

## Джерела

1. Першоджерело

2. Наша пропозиція

3. Реалізація мейнтейнера

4. Реалізація мейнтейнера

5. Докладний кейс

# 12 M7 MCP

**2026-10-29 · Аудит відзначено в релізі**

## Проблема

Я перевіряв конкретний snapshot MCP-сервера: переходи фаз, повторні виклики, збереження стану та передачу роботи. Аудит вимагав відділити дефект контракту від помилки мого спостереження та від обмеження середовища.

## Рішення та роль

Я сформував відтворювані спостереження з причиною, входом і межею висновку. Мейнтейнери створили власний реєстр проблем і реалізували перший maintenance release. Їхні записи не є взаємно-однозначною копією груп мого аудиту.

## Перевірка

Оригінальний публічний PDF збережено без переписування під пізніший реліз. Focused follow-up на зафіксованому commit пройшов 168 тестів і SDK stdio-перевірку, але зберіг три незакриті error-path сценарії. Це не повний повторний аудит.

## Що переношу далі

Аудит має залишати перевірюваний маршрут до помилки та не приховувати те, чого наступний реліз ще не довів.

## Межа зовнішнього результату

Публічне відзначення перевірки й класифікації проблем та випуск мейнтейнерами релізу з виправленнями. Групи аудиту та їхні записи реєстру не взаємозамінні. Прицільна повторна перевірка зберегла незакриті сценарії помилок; живі моделі й IDE не перевірялися.

## Джерела

1. Першоджерело

2. Докладний кейс

# 13 Braintrust SDK JavaScript

**2026-11-02 · Власний PR merged · 2026-11-02T11:37:30Z**

## Проблема

Проблема виникала на межі чанків логування: багатобайтовий UTF-8 символ може бути розділений між двома порціями байтів. Коректний текст не повинен залежати від того, як транспорт розбив потік.

## Рішення та роль

Я зберіг стан decoder між читаннями й відділив завершення потоку від проміжного чанка. Перевірка кожної byte boundary перетворила загальний опис пошкоджених символів на точний інваріант: різні розбиття того самого потоку дають той самий текст.

## Перевірка

Початковий focused прогін мав шість відмов і 13 passing controls. Пов’язані suites пройшли 178 тестів на обох перевірених версіях Node. Це локальні перевірки визначених шляхів, не повний monorepo, browser чи реальний provider end-to-end.

## Що переношу далі

Межа чанка належить транспорту. Стан текстового decoder має належати всьому логічному потоку.

## Межа зовнішнього результату

Злите виправлення потокового логування UTF-8 з локальними регресіями. Повний монорепозиторій, браузер і наскрізний сценарій із реальним провайдером не перевірялися.

## Джерела

1. Першоджерело

2. Докладний кейс

# 14 Kraft

**2026-11-02 · Власний PR merged · 2026-11-02T12:08:30Z**

## Проблема

Повторний user-scope init бачив уже наявну MCP-реєстрацію й завершувався раніше, ніж оновлював skills. Команда виглядала повторюваною, але її друга частина фактично не виконувалася.

## Рішення та роль

Я розділив успішний стан уже наявної реєстрації та справжню помилку subprocess. Перший дозволяє перейти до оновлення skills; друга має зупинити процес до записів. Це виправлення порядку дій, без виклику живого агента.

## Перевірка

Focused suite пройшов 16 тестів. Репозиторні ci-checks та mutation checks перевіряли розрізнення гілок. Claude subprocess був test double; користувацькі конфігурації й зовнішній агент у цих перевірках не змінювалися.

## Що переношу далі

Повторний init має повторювати корисну роботу, а не плутати idempotent стан із відмовою.

## Межа зовнішнього результату

Злите виправлення повторної користувацької ініціалізації. У тестах підпроцес Claude замінено імітацією: без живих викликів агента чи змін користувацьких конфігурацій.

## Джерела

1. Першоджерело

2. Докладний кейс

# 15 PyTorch TensorDict

**2026-11-01 · Внесок у merged fix мейнтейнера**

## Проблема

Частковий checkpoint у вкладеному TensorDictParams міг відмовити навіть із strict=False. Я перевіряв не лише пропущений ключ, а й вкладені пропуски, assignment і pre-hooks - поведінку, на яку вже покладаються користувачі PyTorch.

## Рішення та роль

Я запропонував loader fix і differential controls проти native nn.Module. Мейнтейнер обрав менший owner-level крок: делегувати власному loader PyTorch замість підтримки копії. Публічний credit стосується попереднього fix, report і thorough tests.

## Перевірка

Наш кандидат мав 44 failing сценарії та чотири passing controls на базі; focused suites після нього дали 251 pass і два CUDA skips. Ці числа не описують фінальну реалізацію мейнтейнера. Її прийняття фіксує окремий upstream PR.

## Що переношу далі

Коли контракт уже належить нижчому шару, делегування може бути сильнішим за точнішу копію.

## Межа зовнішнього результату

Мейнтейнер відзначив попереднє виправлення, звіт і ретельні тести. Наш PR #3426 замінено, не злито; менша прийнята реалізація належить мейнтейнеру.

## Джерела

1. Першоджерело

2. Наша пропозиція

3. Відзначення внеску

4. Докладний кейс

# 16 BlackJAX

**2026-11-02 · Власний PR merged · 2026-11-02T14:43:52Z**

## Проблема

Barker MCMC міг відхилити крок до області вищої ймовірності через переповнення проміжного exp. Кінцевий log-expression лишався математично скінченним, але прямий порядок обчислення втрачав його.

## Рішення та роль

Я замінив нестабільний log(1+exp(x)) на стандартний softplus. Важливий доказ - не просто відсутність inf, а правильна proposal probability відносно незалежного обчислення густини. Звичайні значення залишилися контрольними випадками.

## Перевірка

Незалежне обчислення NumPy на 768 CPU-кроках знаходило 142 розбіжності на базовій версії й нуль після зміни; всі 384 звичайні контрольні випадки проходили й раніше. Повний набір тестів не запускався; незмінений збій CI GIST відтворено на базовій та зміненій версіях. Роботу на GPU/TPU та приріст швидкодії не заявлено.

## Що переношу далі

Стабілізувати треба проміжний розрахунок, зберігши математичну величину та контрольні режими.

## Межа зовнішнього результату

Злите виправлення чисельної стабільності Barker MCMC з порівняльними перевірками на CPU. Повний набір тестів не запускався; незмінений збій CI GIST відтворено на точних базовій і зміненій версіях. Роботу на GPU/TPU та приріст швидкодії не заявлено.

## Джерела

1. Першоджерело

2. Докладний кейс

# 17 TensorLy

**2026-11-03 · Власний PR merged · 2026-11-02T23:54:45Z**

## Проблема

Добуток Khatri-Rao з одним множником повертав матрицю до застосування ваг стовпців і маски рядків. Такий шлях можливий і після skip_matrix; він мовчки порушував контракт операції та міг впливати на зважені CP-градієнти.

## Рішення та роль

В обох реалізаціях tenalg додано застосування weights і mask перед поверненням єдиного множника. Після рев’ю JeanKossaifi додані тести скорочено до одного компактного прикладу з буквальним очікуваним результатом. Мейнтейнер прийняв наш PR без заміни реалізації власним виправленням.

## Перевірка

Прямокутна матриця [[1,2,3],[4,5,6]], ваги [2,0,-1] і маска [0,2] мають дати [[0,0,0],[16,0,-12]]. Регресія падає на точній базовій версії в чотирьох режимах NumPy/PyTorch та core/einsum і виявляє окреме вилучення ваг або маски. Обидва відповідні модулі перед поданням пройшли по 70 тестів у кожному режимі; Black перевірив 156 файлів. Нових runtime-перевірок під час запису merge не виконували.

## Що переношу далі

Один добре підібраний приклад може перевіряти обидва обов’язкові множники без великої сітки майже однакових тестів. Вимоги мейнтейнера до простоти стали частиною прийнятого рішення.

## Межа зовнішнього результату

Наш PR злитий. Компактну регресію й два відповідні модулі перевірено на CPU з NumPy/PyTorch та core/einsum: по 70 тестів у кожному режимі. Повний репозиторій, GPU та решту матриці backend/version не запускали; попередню велику oracle-кампанію після скорочення тестів не повторювали.

## Джерела

1. Першоджерело

2. Відзначення внеску

3. Докладний кейс

# 18 Apache TVM-FFI

**2026-11-03 · Внесок у merged fix мейнтейнера**

## Проблема

Рівні короткі byte-значення з inline та heap storage отримували різні structural hashes. Це порушувало очікування StructuralKey cache lookup і native map deduplication. Мій patch виправляв обидва шляхи структурного hashing.

## Рішення та роль

Мейнтейнер публічно пов’язав нашу пропозицію з наступним PR та обрав інший шар: короткий moved std::string одразу створює inline String/Bytes. Набори змінених файлів різні; прямого перенесення мого патчу у порівняних diff не виявлено. Наступний PR #898 злитий 02.11.2026 о 22:16:16 UTC; наша пропозиція лишається superseded.

## Перевірка

Зіставлено нативні стани PR, chronology, коміти та diff. Constructor-side fix прибирає producer path, який використовував початковий reproducer. Наступну реалізацію та всі explicit external-heap випадки для цього запису повторно не виконували.

## Що переношу далі

Відтворення може привести до іншого виправлення. Superseded та merged - різні зовнішні наслідки.

## Межа зовнішнього результату

Мейнтейнерська заміна #898 злита; наш #897 не злито. Виправлення змінює конструктор короткого std::string rvalue, а не переносить наш патч хешування. Повну поведінку заміни й усі сценарії явного heap storage тут повторно не виконували.

## Джерела

1. Першоджерело

2. Наша пропозиція

3. Відзначення внеску

4. Докладний кейс

# 19 torchode · backsolve adjoint

**2026-11-06 · Власний PR merged · 2026-11-06T10:56:19Z**

## Проблема

Backsolve adjoint зупинявся на першій точці спостереження. Якщо вона лежала після початкового часу вздовж інтегрування, градієнт втрачав внесок початкового інтервалу.

## Рішення та роль

Додали зворотне інтегрування від першої точки до початкового часу. Для елементів батчу, які вже на початковому часі, збережено їхній стан.

## Перевірка

Власний PR #56 злитий. Публічний опис містить аналітичний приклад і шість варіантів регресії. Власник підтвердив включення в 1.1.0, доступну на PyPI.

## Що переношу далі

Правильне значення розв’язку може супроводжуватися помилковим градієнтом. Для adjoint потрібно перевіряти весь часовий інтервал.

## Межа зовнішнього результату

Злиття і реліз підтверджено. Локальні прогони описані в PR і датовані роботою над виправленням.

## Джерела

1. Першоджерело

2. Докладний кейс

# 20 Lineax · reshaping identities

**2026-11-09 · Власний PR merged · 2026-11-09T21:40:44Z**

## Проблема

Матеріалізація композиції могла викинути тотожний оператор, що змінює структуру PyTree, хоча кількість елементів збігається.

## Рішення та роль

Скорочуємо тотожний оператор лише зі збіжними вхідною й вихідною структурами. Те саме обмеження діє для згортання до структурованої форми.

## Перевірка

Власний PR #268 злитий. Рев’юер зафіксував додаткові крайові випадки для окремого продовження.

## Що переношу далі

Розмірність і структура PyTree задають різні вимоги до взаємозамінності операторів.

## Межа зовнішнього результату

Прямий merge для описаного класу композицій. Подальші випадки матеріалізації залишаються предметом окремого виправлення.

## Джерела

1. Першоджерело

2. Докладний кейс

# 21 NumPyro · GaussianStateSpace variance

**2026-11-09 · Власний PR merged · 2026-11-09T10:13:37Z**

## Проблема

Відновлення осі часу через swapaxes переставляло батч-осі дисперсії. Для прямокутного батчу це могло викликати помилку broadcasting.

## Рішення та роль

Перенесли вісь часу через moveaxis зі збереженням порядку решти осей. Додали батч (2, 3) із нетривіальною матрицею переходу до наявного набору тестів.

## Перевірка

Власний PR #2405 злитий у master. Регресія перевіряється через наявні тести дисперсії та вибірок.

## Що переношу далі

Нерівні розміри батч-осей і різні параметри допомагають помітити як помилку форми, так і тиху перестановку значень.

## Межа зовнішнього результату

Прямий merge виправлення дисперсії. Зміни параметричних батчів RecursiveLinearTransform обговорюються окремо.

## Джерела

1. Першоджерело

2. Докладний кейс

# 22 CodinGame - Mad Pod Racing

**2026-10-21 (Europe/Helsinki) · Датований зріз арени · UTC 2026-10-20T22:06:15.975Z · 86/1657 (Legend League)**

## Проблема

Я будував рішення для безперервної арени, де локальна перемога не гарантує зовнішнього місця. Для розбору збережено дві історії: швидке відхилення гіпотези й умовний локальний кандидат після зміни критеріїв.

## Рішення та роль

V79 зупинено після 2 перемог у восьми завершених іграх: потрібна межа вже стала недосяжною. V89b дав 221/400, але критерії змінювалися після попередніх відмов, тому я не прирівнюю це до чистої незалежної перемоги чи поданої версії.

## Перевірка

Зовнішній league snapshot має окреме публічне джерело та датований screenshot. Локальна витяжка не містить candidate code, referee і повних traces; самі hashes не дозволяють повторити матчі чи незалежно засвідчити їхній вміст.

## Що переношу далі

Стоп-критерій захищає від доведення бажаного результату. Локальний експеримент і зовнішня арена мають різні одиниці доказу.

## Межа зовнішнього результату

Датований зріз місця в лізі, не фінальний результат турніру й не найкраще місце за весь час.

## Джерела

1. Першоджерело

2. Профіль арени

# 23 Zoria 2026 / UA SKILLS

**2026-10-19 (Europe/Helsinki) · Історичний попередній результат · UTC 2026-10-20T22:06:05.463Z · 5/9 (загальний попередній підсумок учасників другого етапу)**

**2026-10-27 (Europe/Helsinki) · Призер сезону; точне місце не встановлено**

## Проблема

Цей кейс зберігає дві часові точки одного турніру. Попередня таблиця другого етапу та пізніше оголошення організатора описують різні рівні завершеності. Я не додаю їх як два незалежні турнірні результати.

## Рішення та роль

Моя інженерна роль полягала в розробці й перевірці власного ігрового рішення та збереженні меж зовнішнього доказу. Інженерний розбір коду ще не опубліковано, тому книга не підміняє його припущеннями про алгоритм чи автономність.

## Перевірка

Попередня таблиця не включала R3. Пізніше організатор назвав обліковий запис серед призерів. Дата перевірки оголошення не є датою його публікації; точне фінальне місце й виплата цим записом не встановлені.

## Що переношу далі

Історія повинна зберігати попередній стан, не перетворюючи його на ще одну перемогу.

## Межа зовнішнього результату

Публічна таблиця не враховує епізод R3; цей запис не встановлює фінального місця, нагороди або виплати.

Організатор називає цей обліковий запис серед п’яти призерів. Запис не встановлює точного фінального місця, кількості ранжованих учасників або виплати.

## Джерела

1. Першоджерело

2. Першоджерело

# Як зростає журнал

Для нового результату я зберігаю дату спостереження UTC, свою роль, первинне джерело й точний зовнішній статус. Старе відтворення лишається датованим; повторну перевірку не видаю за повний аудит. Додаю запис до results.json, розбір до book/narrative.json і запускаю збірку та офлайн-перевірку.

## Джерела збірки

results.json:
c80354a0d57e05e72de75e4eb34915742bbe64e390295094195ddcd620b68d44

book/narrative.json:
5cd9d11df17f0c585d84be7760e1c40decad8d5539e46406954e3861689134db

pr-status.json:
f6584f2c99261bf8f036c34c09ef6858b7952c8e0f90b692bc39939ae2add7c1

Детальні двомовні звіти та незмінений історичний аудит M7 доступні в репозиторії.

github.com/DiadkoShmek/engineering-results